Software Technical Audit Report

Science Adventure — offline Flutter science game for children (Android)

63 / 100 Acceptable

Application: `science_adventure` 0.3.0+3 · package `com.scienceadventure.app`

Commit audited: `9fbdaf4` on `main` · **Date:** 2026-10-04

Mode: STRICT READ-ONLY. No source, build, manifest or dependency file was modified.

Roles applied: Software Architect · Android/Flutter · Web · Backend · DB · Security · UI/UX · QA · DevOps · Code Review

Companion documents: Future_System_Architecture.pdf · Issues_Register.md · Security_Findings.md · Test_Results.md · Architecture_Diagrams.md · Development_Roadmap.md · Executive_Summary.md

Contents

1. Executive Summary
2. Project Overview
3. Technology Stack
4. Current Architecture
5. Code Quality Review
6. Security Review
7. UI/UX Review
8. Performance Review
9. Functional Test Results
10. Testing Review
11. Dependency Review
12. Strengths
13. Weaknesses
14. Critical Issues
15. Technical Debt
16. Issues Register
17. Score
18. Recommendations
19. Final Technical Assessment
20. Appendix: Method, Evidence and Git Safety

1. Executive Summary

Science Adventure is a well-intentioned, privacy-respecting **offline** learning game. The project is called an "Android app", but it is actually built with **Flutter**: 83 Dart files and ≈22,400 lines in `lib/`, plus a thin Kotlin host. The code compiles cleanly, all **203 automated tests pass**, a debug APK builds, and the APK requests **no Android platform permissions**. Security and privacy are the strongest areas.

The main weaknesses are structural. A single machine holds the toolchain and there is no CI. One 1,391-line widget runs most of the gameplay. State is untyped JSON maps. The whole save (every child, including base64 photos) is rewritten as one Hive value every 10 seconds and on every interaction. These issues do not hurt today's single-device, offline use much, but they are the main **blockers** to turning the product into Android + Web + Backend + Central Database with sync.

> **Verdict:** Overall **63/100 (Acceptable)**. Recommendation **B — Refactor the existing application**, not a rewrite. Flutter already targets the web, so the game can be reused. The data layer must be rebuilt (typed models + SQLite + outbox) *before* backend and sync work starts.
> **Issues:** 0 Critical · 4 High · 8 Medium · 8 Low · 2 Info.

2. Project Overview

Item	Observation

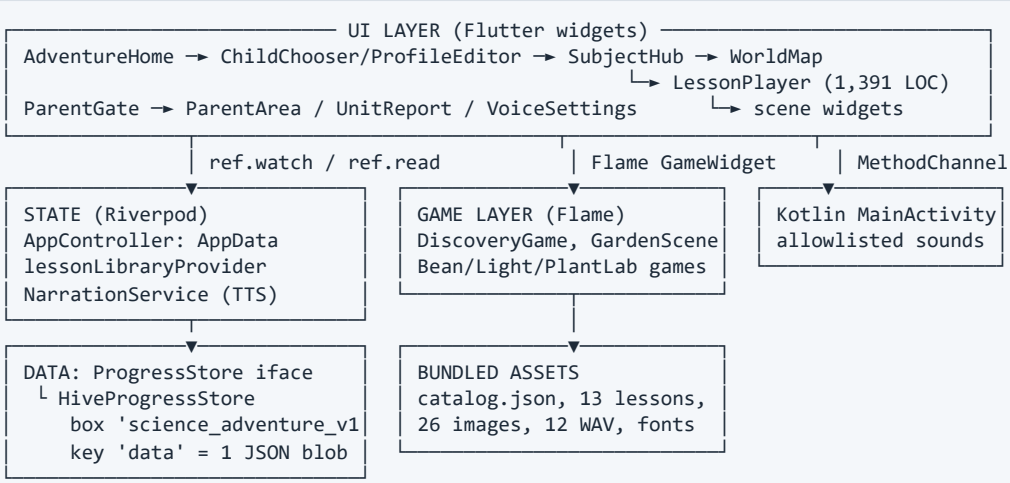
Purpose	Science curriculum game (based on the parent-supplied book "Science Made Simple") for young children. It teaches through interactive scenes: alive/not alive, plants, ourselves, materials, forces, sound, electricity and magnets, Earth and space.
Content	4 islands · 13 playable lessons (9 units + 4 quizzes) in `assets/content/playable/*.json` (436 KB), validated against `schemas/lesson.schema.json` by Python scripts
Users	Children (multiple local profiles) and parents (parent area behind a multiplication gate)
Platforms	Android (minSdk 24, targetSdk 36, landscape-locked). A web build exists for previews only. No iOS project.
Data	Local only: child profile (first name, gender, avatar or photo, palette), settings, assessment attempts, in-progress lesson "journeys", and daily usage seconds
Connectivity	None. No INTERNET permission. TTS uses offline voices only.
Repo size	324 tracked files. Git pack 83.5 MB (dominated by images). 31 design and review documents in `docs/`, plus a 705-line `GAME_MASTER_INSTRUCTIONS.md`
History	17 commits between 2026-09-24 and 2026-10-04 (rapid, feature-heavy iterations)

3. Technology Stack

Layer	Technology (exact versions)	Notes
Language / SDK	Dart 3.13.4 · Flutter 3.47.5 (pinned in `.flutter-version`)	Toolchain stored in gitignored `.tools/`
Android host	Kotlin `MainActivity`; AGP 9.1.0; Kotlin 2.4.0; Gradle 9.3.1 (SHA-256 pinned); JDK 21; Java/Kotlin target 17; compileSdk 36; NDK 28.2	Release: R8 minify and resource shrinking on; signing from untracked `key.properties`
State management	flutter_riverpod 3.4.3	`Notifier<AppData>` with a serialised write queue
Persistence	hive 2.2.3 / hive_flutter 1.1.0	One `Box<String>`, one key `data` (JSON)
Game engine	flame 1.38.2	Scenes, sprite/atlas painting
Media	flutter_tts 4.2.5 · lottie 3.6.1 · native MediaPlayer via MethodChannel	Offline voices filtered
Device	image_picker 1.2.3 · path_provider 2.1.6	System photo picker, no storage permission
i18n	flutter_localizations; inline `t(ar, en)` helpers	No ARB files
Fonts / assets	Fredoka, Nunito, Cairo (OFL) · 26 bundled images (WebP/PNG) · 12 WAV	Original AI-generated art (see CREDITS.md)
Tooling	PowerShell build scripts; Python content pipeline (jsonschema)	No CI
Tests	flutter_test (35 files, 203 tests); 1 Python test	No integration_test / goldens
Not present	Network/HTTP, REST, database server, authentication, analytics, crash reporting, DI framework, code generation	—

4. Current Architecture

Style: a **custom feature-folder architecture with Riverpod state**, close to a simplified MVVM. There is a separate `core` layer (models, storage, providers, design system) and feature folders (`features/biology`, `plants`, `science`, `garden`, `world`, `onboarding`, `parent`). It is *not* Clean Architecture. There is no domain or use-case layer, and most gameplay logic and save migrations live inside `StatefulWidget`s.



Criterion	Rating	Reason
Separation of concerns	Fair	Core models and domain rules (`AttemptRecorder`, `addUsage`, unlock rules) are pure and testable. Scene logic, persistence snapshots and migrations sit in widgets.
Modularity	Fair	Feature folders exist, but `lesson_player.dart` imports 20+ features. Dependency direction is UI → everything.
Testability	Good (unit) / Fair (UI)	The `ProgressStore` interface and `MemoryProgressStore` make state testable. The big widgets need full widget tests.
Scalability / extensibility	Weak	Every new scene type adds branches to `LessonPlayer` and `LessonSession`. Content contracts are untyped.
Maintainability	Fair	Readable naming and consistent formatting, but large files and per-unit special cases.

> **Fit for Android + Web + Backend + Central DB? Partially.** Flutter and Riverpod suit the client side well (one codebase for Android and Web). The persistence and domain layers **need refactoring** (typed models, repositories, relational local DB, outbox) before sync. Classification: **Refactoring**, not a partial or full rewrite.

5. Code Quality Review

Aspect	Assessment
Organisation / folders	Clear top-level split (`core`, `content`, `features/*`, `games`, `parent`). Some misplacement: the `LessonPlayer` used by every subject lives in `features/biology`.
Naming	Generally descriptive (`AttemptRecorder`, `gardenAdventureComplete`). Terse private names in places (`_lab`, `_bean`, `_light` as dynamic maps).
Formatting / static analysis	`dart format` style is consistent and `flutter analyze` is clean, *but* only 3 lint rules are enabled (AUD-013).
Immutability	Good. Models are `const` with `copyWith`, and state transitions are pure functions.
Error handling	Weak. 32 `catch (_)` blocks with no logging, and generic "free some space" messages (AUD-009).
Logging	None (no logger, no crash reporting). This is privacy-friendly but makes field diagnosis impossible.
Configuration	Hard-coded constants (timer 10 s, break 5–30 min, volume factors `.5/.35` per unit). Preview routes sit in `main.dart`.

Findings by smell

Smell	Where	Impact	Severity	Suggested fix (not implemented)
God class / large function	`lesson_player.dart` 1,391 LOC; `build()` ≈750 LOC; `science_lab.dart` 1,209; `sound_stage.dart` 1,031	High regression risk, slow onboarding	HIGH	Controller + one renderer per `playScene`, plus a registry
Primitive obsession / untyped data	184 `json['…']` accesses; 23 `Map<String,dynamic>` in LessonPlayer alone	Runtime cast errors; no API contract	HIGH	freezed / json_serializable models
Hard-coded special cases	`'unit_06.step.008'`, per-unit migrations in `session.dart`	Logic grows with every content revision	MEDIUM	Versioned migration registry, data-driven variants
Dead code	`force_creature.dart` (620 LOC, never imported)	Confusion, build size	MEDIUM	Delete
Unused assets	24 tracked images not in `pubspec.yaml` (~45 MB)	Repo bloat	MEDIUM	Git LFS or art archive
Magic numbers	Volume factors, timer periods, image limits, gate ranges	Tuning requires code edits	LOW	Named constants / config
Side effects in build	`narration.configure()` in `build`; `FutureBuilder(future: load())` in build	Redundant work, flicker	LOW	`ref.listen`; cached futures
Workflow debt	`tmp/`: 300+ one-off Python patch scripts and logs	Untracked change process	MEDIUM	Normal commits plus review and CI
Circular dependencies	None found at package level (feature folders import each other one way through LessonPlayer)	—	INFO	—
Unused dependencies	None. All 8 direct dependencies are imported.	—	OK	—

6. Security Review

Safe static review against OWASP MASVS v2, ASVS and the Mobile Top 10, plus local runtime checks (APK permission dump, network capture of the web preview). No exploitation was done. Full detail: `Security_Findings.md`.

Area	Result
Secrets	PASS None found. Signing material is gitignored.
Permissions	PASS No platform permissions in the built APK. Only an AndroidX internal signature permission.
Exported components / deep links / WebView	PASS Launcher activity only, `taskAffinity=""`, no deep links, no WebView
Backup / data extraction	PASS `allowBackup=false`; all domains excluded from cloud backup and device transfer
Native bridge	PASS Sound names allowlisted (prevents path traversal); volume clamped
Photo handling	PASS Re-encoded, metadata stripped, size and pixel caps; temp file deleted only inside the app cache
Network	PASS No network code; all 98 preview requests were local; offline TTS voices only

Local storage	LOW Child PII in plaintext Hive inside the sandbox (SEC-01)
Resilience	LOW Dart obfuscation not yet configured (SEC-02)
Parent gate	LOW Same problem after failure, unlimited tries (SEC-03)
Input validation	INFO Name accepts markup but is rendered inertly (SEC-05). Validate when data leaves the device.
Auth / sessions / tokens	Not applicable (no accounts). Designed in the future-architecture report.

Security score 17/20. The deductions are hardening items, not vulnerabilities.

7. UI/UX Review

Sources: 94 test-generated screenshots in `tmp/` (1280×800 and 360-wide variants) and live use of the web preview at phone width.

Aspect	Observation	Rating
Overall design	Warm, consistent cream and teal palette; original illustrated scenes of high quality; friendly owl guide (Zizo/Zizi)	Strong
Navigation	Clear hierarchy (Child → Subject → Island → Lesson → Step). Back and Home always visible. Step counter "n/13". Island locking explained.	Good
Typography	Fredoka headings, Nunito body, Cairo Arabic; large sizes. Long titles truncate at phone width.	Good
Touch targets	Large buttons and hotspots with a "touch cue" hand icon, suitable for small fingers	Strong
Accessibility	Semantics labels present throughout (screen reader read every control). Text-scale branches exist. The disabled "Check" button has low contrast. Pre-readers depend on English TTS.	Fair–Good
Feedback	Gentle hints without penalties ("Hmm, let's look again..."), explanations after answers, stars for effort	Strong
Empty / error states	Good parent empty states ("no attempt yet does not mean the child doesn't understand"). Error messages are misleading ("free some space" for code errors).	Mixed
Localisation	Parent area bilingual (Arabic RTL / English). Child area English only. Some dialogs Arabic only.	Weak
Layout / spacing	Challenge screens leave ~40 % empty space at the bottom on tablets; the body-parts screen has 16 labels with crossing lines (cognitive load)	Fair
Consistency	Duplicate heading/subtitle on the hub; the parent-gate owl always says "Zizo"	Fair

Recommended improvements (not applied): a decision on child-facing language plus ARB; scale challenge content to the available height and anchor "Check" near the content; disabled-button contrast ≥ 3:1; ellipsis plus a tooltip for long titles; gender-aware gate mascot; reduce the body-parts matching to batches of 4–6; dedicated, accurate error messages.

8. Performance Review

Area	Finding	Measured?
Startup	Opens Hive, decodes the full blob and validates it, then loads the catalog. The lesson library (13 JSON files, 436 KB) loads eagerly when the map opens.	NOT MEASURED
Memory	26 images, 43.7 MB on disk, ≈156 MB decoded RGBA if all cached. `Flame.images` global cache never evicted.	Static estimate only; runtime NOT MEASURED
Disk I/O / main thread	Every interaction and every 10 s: decode the previous blob + encode the full blob (all children, base64 photos) + flush, on the UI isolate	NOT MEASURED

Rendering	Large atlases drawn with `FilterQuality.high`; futures re-created in `build`	NOT MEASURED
Native audio	`MediaPlayer.prepare()` synchronous on the main thread (small WAVs)	NOT MEASURED
Network	None	—
APK size	Debug APK 208.2 MB (kernel blob 70 MB, 3 ABIs × libflutter ≈107 MB, assets 45.7 MB). The release AAB will be far smaller.	Debug measured; release NOT MEASURED
Memory leaks	Controllers and observers are disposed in the reviewed widgets; MediaPlayer is released on stop. No leak was proven.	NOT MEASURED

9. Functional Test Results

Run on the prebuilt web preview (served read-only on localhost) because no Android device or emulator was available. Details: `Test_Results.md`.

ID	Feature	Status
FT-01–06	Launch, onboarding (empty, long and script-like name), gender/avatar, persistence after restart	PASS
FT-07	Subject hub (duplicate heading, mid-word wrap)	PARTIAL
FT-08	Island / level locking	PASS
FT-09	Lesson list (title truncation)	PARTIAL
FT-10–11	Gameplay: hint on a wrong answer, explanation and progress on a correct one	PASS
FT-12–14	Parent gate (wrong, empty, Arabic digits), parent empty states	PASS
FT-15–16, 18	Mid-activity persistence; local-only network traffic; no console errors	PASS
FT-17	**Delete active child on web → "Could not save" (AUD-005)**	FAIL
FT-19–24	Duplicate names, large data, photo picker, TTS, break reminder, Android back	NOT TESTED

Totals: 15 PASS · 2 PARTIAL · 1 FAIL · 6 NOT TESTED.

10. Testing Review

Type	Present?	Notes
Unit tests	Yes	Models, usage splitting, migrations, catalog integrity, narration with a fake engine
Widget tests	Yes (majority)	Scene flows, navigation, progression, scoring retention, touch interactions
Integration tests (device)	No	No `integration_test/`
Golden / visual regression	No (screenshots written to `tmp/` without comparison)	Side-effect files; no assertion
Database / storage tests	Partial	Round-trips through `MemoryProgressStore`; no Hive corruption tests
Web platform tests	No	Missed AUD-005
CI	No	Tests run manually; results stored in `tmp/*.log`
Result now	**203/203 passed in 87 s**	

Future test strategy: a pyramid of pure-domain unit tests (models, migrations, merge rules, property-based sync tests) → repository tests on in-memory SQLite → widget and golden tests per scene → integration tests on 2 device classes (low-RAM tablet, phone) → web tests (`--platform chrome`) → backend unit, e2e and contract tests (OpenAPI) → security tests (auth, BOLA) → k6 load tests for sync. All gated in CI.

11. Dependency Review

Package	Version	Status
flutter_riverpod	3.4.3	Current
flame	1.38.2	Current
hive / hive_flutter	2.2.3 / 1.1.0	Current on pub.dev, but upstream is no longer actively developed (community fork `hive_ce`). Replace in the refactor.
flutter_tts	4.2.5	Current
image_picker	1.2.3	Current (transitive Android/iOS implementations have patch updates)
path_provider	2.1.6	Current
lottie	3.6.1	Current (used for one celebration animation)
Transitive	7 packages behind (cross_file, image_picker_android/ios, jni, material_color_utilities, objective_c, test_api)	Patch / minor only

Exact pinning plus a committed lockfile plus `--enforce-lockfile` is good supply-chain practice. **No CVE claims**: no vulnerability database was queried. Add `osv-scanner` to CI.

12. Strengths

- Exemplary privacy posture for a children's app: zero permissions, no network, no trackers, backups disabled, sanitised photos.
- Clean static analysis, consistent formatting, immutable models, and a serialised save queue that publishes state only after the write succeeds.
- A meaningful assessment design: first outcome retained, free play and predictions not graded, effort stars kept separate from understanding.
- 203 passing behaviour-focused tests, and a content-validation pipeline with JSON Schema and answer-key traceability.
- High-quality original art, large touch targets, semantic labels, gentle feedback.
- Careful release hygiene: pinned Gradle checksum, signing outside Git, a release build blocked without a key, R8 enabled.
- Extensive design documentation (decisions, reviews, remediation logs).

13. Weaknesses

- Toolchain on a single machine, no CI, ad-hoc patch-script workflow.
- God classes and untyped JSON state; migrations embedded in UI code.
- Monolithic save blob rewritten very frequently; no corruption recovery.
- Heavy assets with an unbounded image cache; release build never produced or tested on a device.
- Swallowed exceptions and misleading errors; no logging.

- Child UI English only; localisation not systematic.

14. Critical Issues (must be addressed before a production release)

No CRITICAL-severity defects. The following HIGH and MEDIUM items should be resolved before a public release:

1. **AUD-001**: reproducible build, CI and a release signing key with backups.
2. **AUD-004 / AUD-006**: save-data resilience (last-good backup, per-child parsing), so a corrupt record cannot lock a child out.
3. **AUD-005**: delete-child failure on web (before any web launch).
4. **AUD-008**: memory profiling on a low-RAM tablet, and asset optimisation.
5. **REC-04**: release AAB with obfuscation, tested on physical devices.

15. Technical Debt

Debt item	Interest (cost if ignored)
Untyped content and journey state	Every new scene adds string keys; sync and backend contracts impossible to validate
LessonPlayer monolith	Change cost and regression risk rise with each unit
Hard-coded per-unit migrations	Must be replicated server-side once synced
Single-blob Hive storage	Blocks partial updates, sync and scaling to many children
Assets in Git and lossless WebP	Slow clones, large APK
No CI and local-only toolchain	Bus factor of 1; "works on my machine" risk
Inline strings	Expensive to localise later

16. Issues Register

ID	Issue	Category	Sev.	Type	Location	Effort
AUD-001	Non-reproducible build, no CI	DevOps	HIGH	Confirmed	repo, scripts/	M
AUD-002	LessonPlayer god class	Code quality	HIGH	Confirmed	lesson_player.dart	L
AUD-003	Untyped Map<String,dynamic> state	Architecture	HIGH	Confirmed	content/, session.dart	L
AUD-004	Full-blob save incl. photos, every 10 s and every interaction	Perf / Data	HIGH	Confirmed (impact not measured)	storage.dart, main.dart	L
AUD-005	Web delete-child failure (`nextInt(1<<32)`)	Bug	MEDIUM	Confirmed	providers.dart:53	S
AUD-006	No save-corruption recovery	Reliability	MEDIUM	Confirmed	main.dart, models.dart	M
AUD-007	Hard-coded migrations in UI / session	Maintainability	MEDIUM	Confirmed	session.dart	M
AUD-008	Heavy images, unbounded cache	Performance	MEDIUM	Confirmed / Potential OOM	assets, Flame.images	M

AUD-009	Swallowed exceptions, misleading errors	Error handling	MEDIUM	Confirmed	32 sites	M
AUD-010	Test gaps and tests writing files	Testing	MEDIUM	Confirmed	test/	M
AUD-011	No localisation system	UX / i18n	MEDIUM	Confirmed	all screens	L
AUD-012	Repo bloat, dead code	Maintainability	MEDIUM	Confirmed	assets/images, force_creature.dart	S
AUD-013	Minimal lint rules	Code quality	LOW	Confirmed	analysis_options.yaml	S–M
AUD-014	Preview routes in production entry point	Code quality	LOW	Confirmed	main.dart	S
AUD-015	Weak parent gate	Child safety	LOW	Confirmed	parent_gate.dart	S
AUD-016	Side effects and futures in build()	Code quality	LOW	Confirmed	main.dart, illustrated_art.dart	S
AUD-017	Plaintext local PII (mitigated)	Security	LOW	Confirmed	storage.dart	S–M
AUD-018	UI polish defects	UI/UX	LOW	Confirmed	hub, map, gate	S–M
AUD-019	copyWith cannot reset voice	Code quality	LOW	Confirmed	models.dart	S
AUD-020	MediaPlayer.prepare on main thread	Performance	LOW	Potential	MainActivity.kt	S
AUD-021	hive upstream maintenance	Dependencies	INFO	Potential	pubspec.yaml	—
AUD-022	README links an untracked APK	Docs	INFO	Confirmed	README.md	S

Full evidence, impact and recommendations for each row are in `Issues_Register.md`, along with recommendations REC-01…REC-06.

17. Score

Category	Score	Bar	Reason
Code Quality	11 / 20		Clean analyzer, immutable models, good naming in core; undermined by god classes, untyped maps, dead code, swallowed errors, weak lint set
Architecture	8 / 15		Sensible Riverpod and feature split with a storage interface seam; no domain or repository layer; logic and migrations in widgets; single-blob persistence
Security	17 / 20		Zero permissions, no secrets, no network, backups off, sanitised photos, allowlisted native bridge; minor hardening left
UI / UX	11 / 15		Excellent art, large targets, semantics, gentle feedback; localisation, polish defects, misleading errors
Performance	5 / 10		Heavy assets with no eviction; frequent full-blob writes on the UI isolate; nothing measured on a device

Testing	6 / 10		203 passing behavioural tests; no CI, integration, golden or web tests; side-effect screenshots
Maintainability	5 / 10		Rich docs, but single-machine toolchain, patch-script workflow, hard-coded migrations, repo bloat
Overall	**63 / 100**	**Acceptable (60–69)**	

18. Recommendations

1. CI and a reproducible toolchain; protect the release key (AUD-001).
2. Revise the privacy decision before accounts and sync; parent-owned accounts; no photo sync by default (REC-01/03).
3. Fix AUD-005 and add web and provider tests.
4. Save resilience and accurate, privacy-safe logging (AUD-006/009).
5. Typed domain models and repositories (AUD-003).
6. Decompose LessonPlayer; migration registry (AUD-002/007).
7. SQLite (drift) plus an outbox, with a zero-loss local migration (AUD-004).
8. Asset optimisation and device profiling (AUD-008).
9. OpenAPI contract and sync design, then NestJS + PostgreSQL + email auth.
10. Optional accounts → sync → Flutter Web, each with a security review.

19. Final Technical Assessment

> **Decision: B — Refactor Existing Application.**
> **Why not A (continue as is):** the data layer cannot support sync, and the god classes will slow every feature.
> **Why not C or D (rewrite):** the UI, game scenes, art, content pipeline and 203 tests are valuable and working. Flutter already compiles to the web, so a rewrite would throw away that working code and gain almost nothing. The problems are concentrated in two areas (persistence and the lesson runtime) that can be replaced step by step behind interfaces (`ProgressStore` already exists).
> **Readiness for production (offline Android):** close, after Phase 0–1 items (release build, CI, data resilience, device testing).
> **Readiness for Android + Web + Backend:** not yet. Phase 2 (typed models + local DB) is the critical path.

Appendix A: Method, Evidence and Git Safety

Step	Detail
Initial git status	Branch `main` @ `9fbdaf4`; one pre-existing untracked file `docs/LATEST_TEACHER_REVIEW_2026_10_04.md` (not created by the audit)
Static review	All core files read in full; large files sampled; grep metrics (catch blocks, dynamic maps, step-ID literals, image loading, semantics)
Dependency restore	Owner approved `flutter pub get --enforce-lockfile`; `pubspec.lock` hash unchanged
Commands run	`flutter analyze`, `flutter pub outdated`, `flutter test`, `flutter build apk --debug`, `aapt2 dump`, `apksigner verify`, `unzip -l`
Runtime	Prebuilt web preview served read-only on 127.0.0.1:8799 (helper script in `reports/_scratch/`), driven via the built-in browser; test data deleted afterwards
Generated side effects	Gitignored only: `build/`, `.dart_tool/`, 94 test screenshots in `tmp/`; global Pub cache populated
Not done (by rule)	No source edits, no formatting, no fixes, no commits, branches, stash, reset or push
Final git status	Unchanged tracked files; untracked: the pre-existing doc + `reports/`

SOURCE CODE MODIFIED: NO